

PE-EC703A — Embedded System

Semester VII • 3L:0T:0P • 3 Credits

Module 3 Exam Notes: I/O Types & Interrupt Service Mechanism

CGEC | B.Tech ECE (2023–27) | *Biraj Sarkar*

Contents

1	Serial and Parallel Communication Ports	2
1.1	UART (Universal Asynchronous Receiver/Transmitter)	2
2	Timer and Counting Devices	3
2.1	8051 Timer Modes	3
3	Watchdog Timer	3
3.1	Watchdog Timer Operation	4
4	Real-Time Clock (RTC)	4
5	Serial Bus Communication Protocols	4
5.1	I2C (Inter-Integrated Circuit) Protocol	5
5.2	CAN (Controller Area Network) Protocol	5
5.3	ISA (Industry Standard Architecture) — Parallel Bus	6
6	Interrupt Service Mechanism	6
6.1	ISR (Interrupt Service Routine)	6
6.2	8051 Interrupt Sources	7
6.3	Different Interrupt Sources (General)	7
7	Multiple Interrupts, Interrupt Latency and Deadline	7
7.1	Multiple Interrupt Handling	8
7.2	Interrupt Latency	8
7.3	Interrupt Deadline	8
7.3.1	Classification of Interrupt Deadlines	9
7.3.2	Key Factors Causing Deadline Violations	9
7.3.3	Design Strategies to Meet Deadlines	9

1. Serial and Parallel Communication Ports

Serial vs. Parallel Communication

Serial Communication: Data is transmitted one bit at a time over a single wire (or differential pair). Slower per line but uses fewer wires, suitable for long-distance communication.

Parallel Communication: Multiple bits (e.g., 8 bits) are transmitted simultaneously over multiple wires. Faster but requires more wires; limited to short distances due to skew and crosstalk.

Feature	Serial Port (UART)	Parallel Port
Wires	2 (TXD, RXD) minimum	8+ data lines + control
Speed	Lower (per port)	Higher
Distance	Long range (RS-232, RS-485)	Short (PCB-level, <1 m)
Cost	Low (fewer pins, simpler hardware)	Higher
Examples	UART, SPI, I2C, USB	8051 P0–P3 ports, old printer port

1.1 UART (Universal Asynchronous Receiver/Transmitter)

- **Asynchronous:** No shared clock line — both devices agree on **baud rate** beforehand.
- **Frame format:** Start bit (1, low) + Data bits (5–9) + Optional Parity bit + Stop bit(s) (1 or 2, high).
- **Baud rate:** Number of signal changes per second (symbols/second). At 9600 baud, 1 bit = 104 μ s.
- **Wiring:** TXD of device A connects to RXD of device B, and vice versa. Ground is shared.

UART Baud Rate Formula (8051)

$$\text{Baud Rate} = \frac{f_{osc}}{32 \times 12 \times (256 - \text{TH1})} \quad (\text{Mode 1 or 3, Timer 1 in 8-bit auto-reload})$$

For 9600 baud at $f_{osc} = 11.0592$ MHz: TH1 = FDH (253)

2. Timer and Counting Devices

Timer / Counter

A **timer** is a hardware counter that increments on each internal clock pulse (counting time). A **counter** increments on each external event pulse (e.g., counting RPM or pulses). Both use the same hardware register; the source of the clock selects timer vs. counter mode.

2.1 8051 Timer Modes

The 8051 has two timers (Timer 0 and Timer 1), each configurable in 4 modes via TMOD register:

Mode	Description	Use
Mode 0	13-bit timer/counter (TH + 5 LSBs of TL)	Legacy 8048 compatibility
Mode 1	16-bit timer/counter (TH:TL = 65536 counts)	Long time delays
Mode 2	8-bit auto-reload (TL counts, TH is reload value)	Baud rate generation
Mode 3	Two independent 8-bit timers (Timer 0 only)	Dual-purpose timing

Timer Delay Calculation (8051, Mode 1)

$$\text{Time delay} = (65536 - \text{Initial Count}) \times \frac{12}{f_{osc}}$$

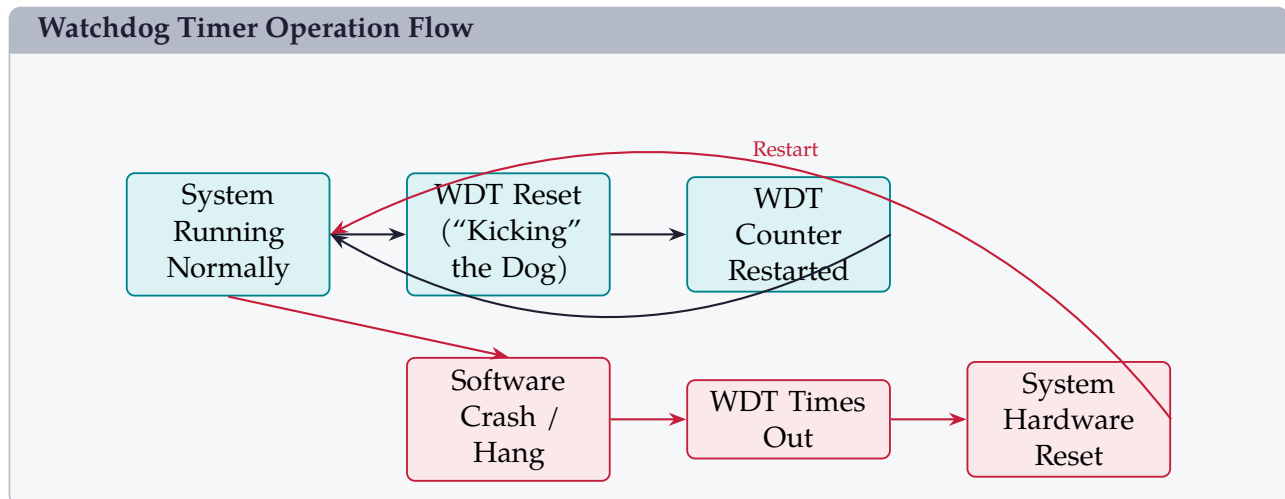
Example: For 10 ms delay at 12 MHz: Initial Count = $65536 - 10000 = 55536 = \text{D8F0H}$

3. Watchdog Timer

Watchdog Timer (WDT)

A **watchdog timer** is a hardware timer that automatically **resets the microcontroller** if the software fails to periodically reset (“kick” or “pet”) it within a defined time interval. It is used to detect and recover from software deadlocks, infinite loops, or crashes in embedded systems.

3.1 Watchdog Timer Operation



- **Kicking (petting) the WDT:** Software writes a specific value to the WDT register before the timeout period expires. This restarts the countdown.
- **WDT Timeout Period:** Configurable (e.g., 100 ms to several seconds). If not reset in this time, the WDT asserts a hardware reset signal.
- **Window Watchdog (WWDT):** Advanced variant — must be kicked *within* a specific time window (not too early, not too late) to detect stuck loops that still kick the WDT.

4. Real-Time Clock (RTC)

Real-Time Clock (RTC)

An **RTC** is a dedicated counter/timer circuit that tracks the current time (hours, minutes, seconds) and date (day, month, year) continuously — even when the main system is powered off — using a separate battery (e.g., CR2032 coin cell) and a 32.768 kHz crystal.

- **Crystal Frequency:** $32.768 \text{ kHz} = 2^{15} \text{ Hz}$. A 15-bit counter overflows exactly once per second.
- **Interface:** Typically I2C or SPI serial interface (e.g., DS3231, PCF8563).
- **Applications:** Data loggers, digital clocks, alarm systems, access control systems.
- **Alarms/Interrupts:** RTC can generate periodic interrupts (every second, minute, etc.) to wake up the MCU from sleep mode.

5. Serial Bus Communication Protocols

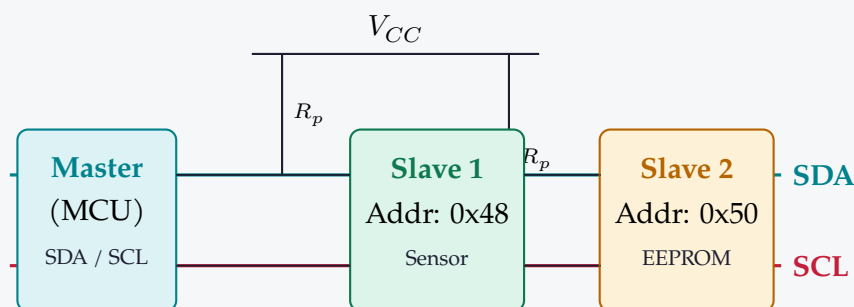
5.1 I2C (Inter-Integrated Circuit) Protocol

I2C Protocol

I2C (pronounced “I-squared-C”) is a synchronous, multi-master, multi-slave, half-duplex serial protocol using only **two wires**: SDA (Serial Data) and SCL (Serial Clock). Developed by Philips (NXP). Each device has a unique 7-bit (or 10-bit) address.

- **Wires:** SDA (bidirectional data) + SCL (clock generated by master). Both are open-drain with pull-up resistors.
- **Speed Modes:** Standard (100 kbps), Fast (400 kbps), Fast-Plus (1 Mbps), High-Speed (3.4 Mbps).
- **Start/Stop Condition:** Start = SDA falls while SCL is high. Stop = SDA rises while SCL is high.
- **ACK/NACK:** After each 8-bit byte, receiver pulls SDA low (ACK) or leaves it high (NACK).
- **Applications:** Connecting sensors (temperature, pressure), EEPROMs, RTC modules, LCD controllers.

I2C Bus — Master–Slave Connection



5.2 CAN (Controller Area Network) Protocol

CAN Bus Protocol

CAN is a robust, multi-master serial bus protocol designed specifically for **automotive and industrial** applications where reliability in high-noise environments is critical. It uses differential signaling on two wires: **CAN-H** (High) and **CAN-L** (Low). No master/slave — all nodes are equal.

- **Differential Signaling:** CAN-H and CAN-L carry inverted signals. Noise affects both equally and is cancelled at the receiver, giving high EMI immunity.
- **Speed:** Up to 1 Mbps (standard CAN), up to 5–8 Mbps (CAN-FD).
- **Message-Based:** No addresses. Each message has an **Identifier (ID)** (11-bit or 29-bit). All nodes receive all messages and filter by ID.
- **CSMA/CA:** Carrier Sense Multiple Access with Collision **Avoidance** — lower ID (higher priority) wins arbitration without destroying the message.

- **Error Detection:** CRC checking, bit stuffing, ACK slots — very robust error handling.
- **Applications:** Automotive ECUs, industrial PLCs, medical devices, robotics.

5.3 ISA (Industry Standard Architecture) — Parallel Bus

ISA Bus

ISA is a **parallel communication standard** developed by IBM (1981) for connecting peripheral cards (sound cards, modems, I/O cards) to the PC motherboard. It uses an 8-bit or 16-bit wide data bus at 8 MHz. Largely obsolete (replaced by PCI, PCIe) but historically significant.

Feature	I2C	CAN	ISA
Type	Serial (synchronous)	Serial (async, differential)	Parallel
Wires	2 (SDA, SCL)	2 (CAN-H, CAN-L)	8/16 data + 20 address + control
Max Speed	3.4 Mbps	1 Mbps (8 Mbps CAN-FD)	8 MHz (16 MB/s)
Distance	Short (PCB-level, <1 m)	Up to 1 km at 50 kbps	Very short (backplane)
Use	Sensors, EEPROMs	Automotive, industrial	Legacy PC expansion cards

6. Interrupt Service Mechanism

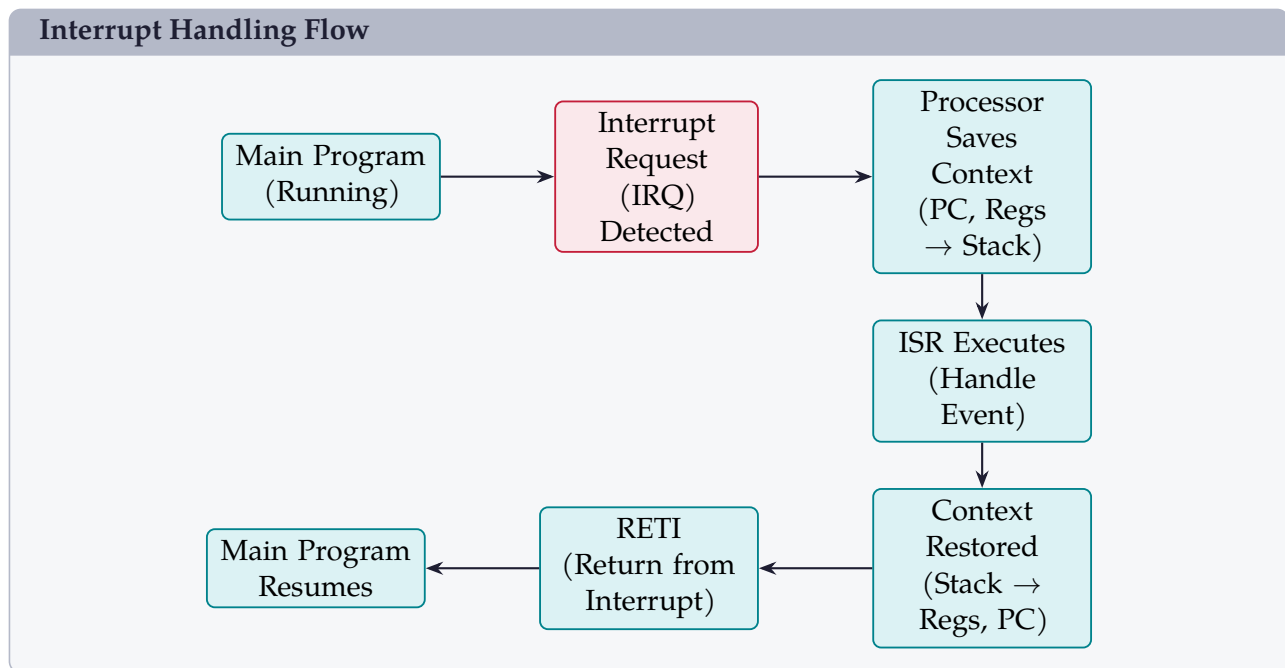
Interrupt

An **interrupt** is a signal sent to the processor by hardware or software indicating that an event requiring immediate attention has occurred. The processor **suspends** its current task, saves its state (PC, registers), and executes a special routine called the **Interrupt Service Routine (ISR)**. After the ISR completes, execution resumes from where it was interrupted.

6.1 ISR (Interrupt Service Routine)

ISR — Interrupt Service Routine

An **ISR** is a software function that is automatically called by the processor when a specific interrupt occurs. It handles the interrupt event (e.g., reads a received byte from UART, clears a timer flag) and must be kept **short and fast** to minimize system latency.



6.2 8051 Interrupt Sources

The 8051 has **5 interrupt sources** with a fixed priority:

Interrupt	Source	Vector Address	Priority (Default)
INT0	External Pin P3.2	0003H	Highest (1)
Timer 0	Timer 0 overflow	000BH	2
INT1	External Pin P3.3	0013H	3
Timer 1	Timer 1 overflow	001BH	4
Serial	UART TI or RI flag	0023H	Lowest (5)

6.3 Different Interrupt Sources (General)

- **External Interrupts:** Triggered by a signal change on a pin (edge-triggered or level-triggered).
- **Timer Interrupts:** Triggered when a timer/counter register overflows.
- **Communication Interrupts:** Triggered when UART receives a byte, or SPI transfer completes.
- **ADC Interrupts:** Triggered when analog-to-digital conversion is complete.
- **Software Interrupts (SWI):** Triggered by executing a special instruction (used in ARM for system calls).
- **Reset:** Non-maskable; forces processor to restart (highest priority).

7. Multiple Interrupts, Interrupt Latency and Deadline

7.1 Multiple Interrupt Handling

When several interrupts occur simultaneously, the system must resolve priority:

- **Fixed Priority:** Each interrupt has a fixed, pre-assigned priority level. Higher priority ISR always preempts lower priority (8051 default behavior via IP register).
- **Nested Interrupts:** A higher-priority interrupt can interrupt a currently-executing lower-priority ISR. Requires saving the lower ISR's state on the stack.
- **Interrupt Masking:** Individual interrupts can be enabled/disabled via IE (Interrupt Enable) register bits to prevent interference during critical code sections.

8051 Interrupt Enable (IE) Register

EA	—	—	ES	ET1	EX1	ET0	EX0
Bit 7	6	5	4	3	2	1	0

EA = Global enable; ES = Serial; ET1 = Timer1; EX1 = INT1; ET0 = Timer0; EX0 = INT0

7.2 Interrupt Latency

Interrupt Latency

Interrupt latency is the time elapsed from when an interrupt signal is asserted to when the first instruction of the ISR begins executing. It includes: detecting the interrupt, completing the current instruction, saving context (pushing to stack), and fetching the ISR vector.

Interrupt Latency Components

$$t_{\text{latency}} = t_{\text{detect}} + t_{\text{inst_complete}} + t_{\text{context_save}} + t_{\text{vector_fetch}}$$

- t_{detect} : Time for hardware to sample and recognize the interrupt signal.
- $t_{\text{inst_complete}}$: Time to finish the currently executing instruction (worst case: longest instruction).
- $t_{\text{context_save}}$: Time to push PC (and registers, if applicable) to stack.
- $t_{\text{vector_fetch}}$: Time to read the ISR address from the vector table and jump.

ARM Cortex-M: Typical latency = 12 clock cycles (with zero wait-state memory).

7.3 Interrupt Deadline

Interrupt Deadline

An **interrupt deadline** is the maximum allowable time window within which an interrupt request must be fully serviced. It starts at the moment the interrupt event occurs (trigger) and ends when the required response action is completely finished.

7.3.1 Classification of Interrupt Deadlines

- **Hard Deadline:** Missing the deadline results in total system failure, physical damage, or safety hazards (e.g., airbag deployment, pacemaker pacing pulse, anti-lock braking systems).
- **Soft Deadline:** Missing the deadline degrades the quality of service (QoS) but does not cause system failure (e.g., dropping a frame in video playback, delaying a UI button highlight).
- **Firm Deadline:** Missing the deadline means the processed data is completely useless, but it does not cause catastrophic damage (e.g., packet arrival in telecommunication where outdated packets are simply discarded).

Real-Time Constraint & Timing Budget

For a system to be real-time compliant, the sum of interrupt latency and the ISR execution time must not exceed the deadline:

$$t_{\text{latency}} + t_{\text{ISR_execution}} \leq t_{\text{deadline}}$$

Where:

- t_{latency} : The time taken to detect the interrupt, save context, and jump to the ISR start.
- $t_{\text{ISR_execution}}$: The execution time of the active instructions inside the ISR.
- t_{deadline} : The timing budget imposed by the physical process/application.

7.3.2 Key Factors Causing Deadline Violations

1. **Interrupt Nesting & Blocking:** If a higher-priority interrupt preempts the current ISR, or if interrupts are globally disabled (during critical sections of the OS), the execution of the lower-priority ISR is delayed, increasing its response time.
2. **Interrupt Jitter:** The variance in interrupt latency caused by processor states (pipeline stalls, cache misses, branch mispredictions, or instruction execution state).
3. **Over-complex ISRs:** Performing heavy math (e.g., floating-point computations, FFT) or blocking calls (e.g., waiting for serial I/O or delays) inside the ISR.

7.3.3 Design Strategies to Meet Deadlines

- **Keep ISRs “Short and Sweet”:** Perform only the bare minimum operations (e.g., reading a register, clearing a flag, placing data in a circular buffer) and signal the main loop or a background task using a volatile flag or RTOS semaphore.
- **Deferred Processing (Bottom Half):** Defer the heavy non-critical execution to the background (often called a deferred procedure call, tasklet, or thread).
- **Proper Priority Assignment:** Use **Deadline Monotonic Scheduling (DMS)**, where interrupts with shorter deadlines are assigned higher priority, regardless of their arrival rate.

Quick Revision — Module 3

I/O Types & Interrupt Service Mechanism | PE-EC703A

Key Formulas at a Glance

UART Baud Rate (8051)	$\text{Baud} = \frac{f_{osc}}{32 \times 12 \times (256 - \text{TH1})}$
Timer Delay (Mode 1)	$t_{delay} = (65536 - \text{Init_Count}) \times \frac{12}{f_{osc}}$
Interrupt Deadline	$t_{latency} + t_{ISR} \leq t_{deadline}$
RTC Crystal Freq	$f = \frac{32768 \text{ Hz}}{\text{overflow/second}} = 2^{15} \text{ Hz (15-bit counter, 1 overflow/second)}$

One-Line Definitions

- **UART:** Asynchronous serial protocol using start/stop bits; configurable baud rate.
- **I2C:** 2-wire synchronous serial bus (SDA + SCL); multi-master, 7-bit addressing.
- **CAN:** 2-wire differential serial bus for automotive/industrial; message-ID arbitration.
- **ISA:** Legacy 8/16-bit parallel expansion bus from IBM PC (1981); obsolete.
- **Timer:** Hardware counter that counts internal clock pulses to measure time.
- **Counter:** Hardware timer counting external event pulses.
- **WDT:** Watchdog Timer — resets MCU if software fails to kick it periodically.
- **RTC:** Real-Time Clock — tracks date and time using 32.768 kHz crystal and coin-cell battery.
- **Interrupt:** Hardware/software signal that suspends main program to run ISR.
- **ISR:** Interrupt Service Routine — short function handling the interrupt event.
- **Interrupt Latency:** Delay from interrupt request to first ISR instruction.
- **Interrupt Deadline:** Maximum allowable time for ISR to complete its response.
- **NVIC:** ARM's Nested Vectored Interrupt Controller; handles up to 240 priority-based interrupts.

Frequently Asked / Viva Questions

Q1. What is the difference between serial and parallel communication?

Serial sends data one bit at a time (fewer wires, longer distance). Parallel sends multiple bits simultaneously (more wires, faster but short-distance only).

Q2. What is UART? Describe its frame format.

UART is Universal Asynchronous Receiver/Transmitter. Frame: **Start bit** (1 bit,

Frequently Asked / Viva Questions (contd.)

low) + **Data** (5–9 bits, LSB first) + **Parity bit** (optional) + **Stop bit(s)** (1 or 2 bits, high). Both ends agree on baud rate; no clock wire.

Q3. What are the two wires in I2C and what do they do?

SDA (Serial Data Line): carries data bidirectionally. **SCL** (Serial Clock Line): carries the synchronization clock generated by the master. Both are open-drain with pull-up resistors.

Q4. Why is CAN used in automobiles instead of I2C or UART?

CAN uses differential signaling which is immune to electromagnetic noise. It supports multi-master arbitration (priority-based, non-destructive), reliable error detection (CRC, ACK, bit stuffing), and distances up to 1 km at 50 kbps — all essential in a noisy vehicle environment.

Q5. What is a Watchdog Timer? Why is it important?

A WDT is a hardware timer that resets the MCU if software fails to periodically reset it. It is critical in embedded systems because it provides autonomous recovery from software hangs, infinite loops, or crashes — without human intervention.

Q6. What is the purpose of the 32.768 kHz crystal in an RTC?

32.768 kHz = 2^{15} Hz. A 15-bit counter driven at this frequency overflows exactly once every second, providing a precise 1-second tick for timekeeping without complex division logic.

Q7. Define interrupt latency and list its components.

Interrupt latency is the time from interrupt assertion to ISR start. Components: interrupt detection time + current instruction completion time + context save time (PC/regs to stack) + vector table fetch time.

Q8. What is a vector table in embedded systems?

A vector table is a fixed memory location (e.g., 0x00000000 in ARM) that stores the starting addresses (vectors) of all ISRs. When an interrupt occurs, the CPU reads the corresponding vector and jumps to the ISR.

Q9. How does the 8051 handle two simultaneous interrupts?

The 8051 uses a fixed priority scheme. INT0 has the highest priority and Serial has the lowest. If two interrupts occur simultaneously, the higher-priority one is serviced first. Priority can also be adjusted using the IP (Interrupt Priority) register to create two priority levels.

Q10. What is the difference between edge-triggered and level-triggered interrupts?

Edge-triggered: ISR fires on signal transition (rising or falling edge). Responds to event occurrence. **Level-triggered:** ISR fires as long as the signal remains at the active level. The ISR keeps being called until the condition is cleared.

Q11. Why should an ISR be kept as short as possible?

Frequently Asked / Viva Questions (contd.)

Long ISRs increase interrupt latency for other interrupts, can miss deadlines, may cause data loss (e.g., UART buffer overflow), and increase system unpredictability. Heavy processing should be deferred to the main loop using a flag set by the ISR.

Q12. What is the Timer Mode 2 in 8051 and why is it used for baud rate generation?

Mode 2 is an 8-bit auto-reload mode where TL counts from the reload value (stored in TH) to 255 and overflows. On overflow, TL is automatically reloaded from TH. This creates a precise, continuous periodic interrupt — ideal for generating UART baud rates without software intervention.

Serial Communication Protocols at a Glance

Protocol	Wires	Speed	Masters	Typical Use
UART	2 (TX, RX)	115.2 kbps typical	1	Debug console, GPS
SPI	4 (MOSI, MISO, SCK, CS)	Up to 50 Mbps	1	Flash memory, LCD
I2C	2 (SDA, SCL)	Up to 3.4 Mbps	Multiple	Sensors, EEPROM
CAN	2 (CAN-H, CAN-L)	Up to 1 Mbps	Multi-master	Automotive ECU
USB	4 (D+, D-, VCC, GND)	Up to 10 Gbps	1 host	PC peripherals